associate the number of items we have in stock: quantities that can be compared to each other, to determine which products are scarcer and most urgent to re-supply.

Now, if we sort the list of entries according to one criterion, for example the name, we need to scan the whole list to find a given value for the other criterion, in this case the quantity in stock.

And if we use a min heap with the scarcer products at its top, then (as we learned in chapter 3) we will also need linear time to scan the whole heap looking for a product to update.

Long story short, none of the basic data structure we describe in appendix C, nor a priority queue, can single-handedly solve our problem.

3.2 Solution: Description and API

Now that we have an idea about what our ideal container should do (but we still don't know how it should do it), we can define an abstract data structure (*ADT*) with the appropriate API: as long as our implementations will aide by this API, we can use any of them seamlessly in an application, or as part of a more complex algorithm, without having to worry about breaking anything.

Table 3.1 API and contract for SortedPriorityQueue

Abstract Data Structure: Sorted Priority Queue	

API	`class SortedPriorityQueue {`
	`  top() → element`
	`  peek() → element`
	`  insert(element, priority)`